

# Crypto Price Tracker — Project Roadmap

## Tech Stack

- **Frontend:** Next.js with TypeScript, Tailwind CSS
- **Backend:** Node.js with Express
- **Database:** PostgreSQL
- **Caching:** Redis
- **Charts:** Recharts
- **Auth:** JWT (access + refresh tokens)
- **API:** CoinGecko free API
- **Hosting:** Vercel (frontend), Render (backend), Supabase (Postgres), Upstash (Redis)

## Week 1 — Setup + Core Dashboard

### Day 1-2: Project setup

- Initialize Next.js + TypeScript project
- Set up Express backend with basic folder structure (routes, controllers, services)
- Connect PostgreSQL database
- Set up Git repo and basic CI (optional)

### Day 3-4: CoinGecko integration

- Build a backend service layer that calls CoinGecko [/coins/markets](#) endpoint
- Set up Redis caching so the backend fetches fresh data every 30 seconds and serves cached data to all users in between (this avoids hitting CoinGecko's rate limit)
- Create the [/api/markets](#) endpoint on our backend that the frontend will call

### Day 5-7: Dashboard UI

- Build the dashboard page — table/grid showing top coins by market cap
- Show price, 24h change, market cap, volume, last updated time
- Add loading states and basic error handling
- Make it responsive for mobile/tablet

**End of Week 1 checkpoint:** Working dashboard showing live top coins, refreshing automatically, no crashes on slow network.

## Week 2 — Coin Details, Search, Sorting

### Day 8-9: Search and sort

- Add search bar (filter by coin name/symbol)
- Add sorting by market cap, price, 24h change, volume
- Add pagination (simple page-based, not infinite scroll — easier to get right in the time we have)

### Day 10-12: Coin details page

- Build dynamic route for individual coin (`/coin/[id]`)
- Show coin logo, current price, rank, market cap, 24h high/low, ATH/ATL, circulating supply
- Add link to official website and explorer (CoinGecko provides these links in their API response)

### Day 13-14: Price chart

- Integrate Recharts
- Pull historical data from CoinGecko's `/coins/{id}/market_chart` endpoint
- Add time filters: 24H, 7D, 30D, 90D, 1Y
- Cache chart data per coin per time range in Redis (chart data doesn't need to update every 30 seconds — cache for 5 minutes)

**End of Week 2 checkpoint:** Users can search, sort, click into any coin, and see a working price chart.

## Week 3 — Auth, Watchlist, Trending

### Day 15-16: Authentication

- Build register/login endpoints with bcrypt password hashing
- Issue JWT access token + refresh token
- Add forgot password flow (basic email-based reset, or simplified version if email service isn't set up)

### Day 17-18: Watchlist

- Add/remove coins from watchlist (requires login)
- Store watchlist in PostgreSQL, linked to user ID
- Show watchlist section on dashboard for logged-in users

### Day 19-20: Trending section

- Add "Top Gainers" and "Top Losers" sections using the 24h change data we already have
- Add "Most Traded" section sorted by volume
- These don't need new API calls — just different sorting of data we're already caching

### Day 21: Profile page

- Basic profile view/edit (name, email)

**End of Week 3 checkpoint:** Full auth flow works, users can build a watchlist, trending sections are live.

## **Week 4 — Admin Panel, Polish, Deployment**

### **Day 22-23: Admin panel (basic)**

- Simple admin-only route (protected by role check in JWT)
- View list of registered users
- View basic app stats (total users, total watchlist entries)
- Skip announcements feature if time is tight — mark as future addition

### **Day 24-25: Polish and error handling**

- Go through every page and check loading states, empty states, error states
- Test on mobile, tablet, desktop
- Fix any UI inconsistencies
- Add proper 404 and error pages

### **Day 26-27: Performance pass**

- Check that Redis caching is actually reducing API calls (log and verify)
- Optimize images (coin logos) using Next.js Image component
- Check page load speed, fix anything obviously slow

### **Day 28-30: Deployment and documentation**

- Deploy frontend to Vercel, backend to Render
- Set up production PostgreSQL on Supabase, Redis on Upstash
- Write README with setup instructions and environment variable list
- Final round of testing on the live deployed version